
SCION: Scene Composition with Instanced Neural Primitives

Supplementary Material

William Koch Amogh Joshi Cyrus Vachha Cheng Zheng Felix Heide
Princeton University
{william.koch, amoghjoshi, cvachha, chengzh, fheide}@princeton.edu

S1 Additional architectural details

S1.1 Initialization details

We initialize instances from the sparse COLMAP point cloud [Schönberger and Frahm, 2016]. The default setting uses all seed points. For each seed position $\mathbf{x}_i$, we project it into the training views, sample frozen DINOv3 features [Siméoni et al., 2025], average the visible samples, reduce them with Principal Component Analysis (PCA), and normalize:

$$\bar{f}_i = \frac{\sum_{v \in \mathcal{V}_i} w_{iv} F_v(\pi_v(\mathbf{x}_i))}{\sum_{v \in \mathcal{V}_i} w_{iv}}, \quad f_i = \frac{\text{PCA}(\bar{f}_i)}{\|\text{PCA}(\bar{f}_i)\|_2}.$$

The default weights are uniform over visible views, and the position weight in this clustering feature is zero. We therefore assign instances to primitives with spherical k -means on DINO features only:

$$\begin{aligned} \{\mathbf{m}_k\}_{k=1}^K, \{a_i\}_{i=1}^N &= \text{sph-kmeans}_K(\{f_i\}_{i=1}^N), \quad \mathcal{I}_k = \{i : a_i = k\}, \\ \mathbf{c}_{kj}^{L,0} &= \frac{1}{|\mathcal{I}_k|} \sum_{i \in \mathcal{I}_k} \text{logit}(r_i) + \epsilon_{kj}^c, \\ \theta_{kj}^{L,0} &= (\epsilon_{kj}^\mu, \mathbf{q}_{kj}^0, \log(\gamma d_{kj}) \mathbf{1}, \text{logit}(0.1), \mathbf{c}_{kj}^{L,0}). \end{aligned}$$

Here r_i is the RGB color attached to the COLMAP point, ϵ_{kj}^μ samples a small local offset around the primitive origin, $\mathbf{q}_{kj}^0$ is drawn from the initialization rotation cone, and d_{kj} is the local nearest-neighbor spacing used to set the initial splat scale. The instance state starts at the point position, identity rotation, zero log-scale, and opacity logit $\text{logit}(0.5)$. DINO is not used as a training loss in the default stack.

S1.2 Standard splat regularizers

The two terms $\mathcal{L}_{\text{scale}}$ and $\mathcal{L}_{\text{opac}}$ in Eq. 5 follow standard 3DGS practice. $\mathcal{L}_{\text{scale}}$ penalizes anisotropic local-frame splat scales beyond ratio ρ :

$$\mathcal{L}_{\text{scale}} = \lambda_s \frac{1}{|\mathcal{A}|} \sum_{r \in \mathcal{A}} \left[\frac{\max_d e^{s_{rd}^L}}{\min_d e^{s_{rd}^L}} - \rho \right]_+,$$

where $\mathcal{A}$ is the active splat set. $\mathcal{L}_{\text{opac}}$ is a one-sided barrier on local opacity logits:

$$\mathcal{L}_{\text{opac}} = \lambda_o \sum_{r \in \mathcal{A}} [o_r^L - \tau_o]_+^2.$$

S1.3 Projected adversarial discriminator

The non-saturating GAN term [Goodfellow et al., 2014] of Sec. 3.2 uses a projected discriminator [Sauer et al., 2021]: a frozen VGG-16 backbone [Simonyan and Zisserman, 2014] followed by light-weight learnable projection heads at each feature scale. Writing D for the discriminator and $x \sim p_{\text{real}}, \hat{x} = \hat{I}_v(\Theta, \Phi^{\text{cont}}) \sim p_{\text{render}}$, the discriminator is trained against

$$\mathcal{L}_D = -\mathbb{E}_x [\log D(\tilde{A}_p(x))] - \mathbb{E}_{\hat{x}} [\log(1 - D(\tilde{A}_p(\hat{x})))] , \quad (\text{S1})$$

while the scene parameters minimize the non-saturating generator term

$$\mathcal{L}_{\text{adv}} = -\mathbb{E}_{\hat{x}} [\log D(\tilde{A}_p(\hat{x}))] , \quad (\text{S2})$$

where $\tilde{A}_p$ applies Adaptive Discriminator Augmentation [Karras et al., 2020] whose probability p is updated to keep D 's accuracy on real samples near a target. D and $(\Theta, \Phi^{\text{cont}})$ are updated alternately at separate cadences. The freeze on the VGG-16 backbone means only $\sim 10^4$ projection-head parameters need to keep up with the generator, which is essential at our compact operating points where a from-scratch patch discriminator saturates.

S1.4 MH splat densification: vocabulary-specific equations

This appendix gives the three equations referenced in Sec. 3.3 that scope the MH-3DGS birth rule [Kim et al., 2025] to the vocabulary. Notation follows Sec. 3.3: $\bar{g}_{kj}$ is the scatter-added screen gradient from Eq. 6, $\bar{\ell}_{kj}$ is the accumulated image L1 residual sampled at the row's projected center, and d_{kj} counts recent visits to a row.

Per-primitive importance score. For each alive row in primitive c_k , we combine opacity, screen gradient, and image residual, z-scored within c_k :

$$q_{kj} = \sigma(w_o z_k(o_{kj}) + w_g z_k(\bar{g}_{kj}) + w_\ell z_k(\bar{\ell}_{kj})) . \quad (\text{S3})$$

Z-scoring within each primitive is important: a template used by many instances receives more total rasterizer signal, but births should depend on where that template is under-fit relative to its own rows.

Acceptance probability. Accepted children are jittered in the primitive-local frame with proposal standard deviation proportional to the parent's local scale and damped by $1/(1 + \log(1 + d_{kj}))$. Coarse and fine proposal batches are annealed over the refinement window. Acceptance also penalizes already dense local regions:

$$p_{\text{acc}} = \sigma(q_{kj} - \tau_{\text{BIRTH}}) \frac{1}{1 + \alpha_v V_k(x^+)}, \quad (\text{S4})$$

where $V_k(x^+)$ counts same-primitive splats in the proposal voxel.

Alpha-compositing mass relocation. When a splat is copied into ν overlapping rows, relocation and birth both preserve alpha-compositing mass:

$$\sigma(o') = 1 - (1 - \sigma(o))^{1/\nu}, \quad \mathbf{s}' = \mathbf{s} - \frac{1}{3} \log \nu. \quad (\text{S5})$$

We apply the correction to the donor and to the new or copied rows.

S1.5 Densification pseudocode

This appendix gives pseudocode for the active densification operators of the default stack in Sec. 3.3. Primitive-level densification is disabled by `candidate_strategy=none`; the primitive set is fixed after initialization. Notation follows Sec. 3: $\mathcal{S}_k$ is the alive splat set of primitive c_k , $\bar{g}_{kj}$ is the accumulated screen-space splat gradient, $\bar{\ell}_{kj}$ is the accumulated projected L1 residual, and $\bar{h}_i$ is the accumulated instance gradient.

Algorithm S1 Primitive-local MH splat densification (one firing).

Require: primitives $\{c_k\}_{k=1}^K$, free row list, accumulators $(\bar{g}, \bar{\ell}, d)$, thresholds $\tau_{splat}, \tau_{BIRTH}$, cap $S_{\max}$

- 1: **for** each primitive $c_k \in \mathcal{C}$ **do**
- 2: $\mathcal{D}_k \leftarrow \{j \in \mathcal{S}_k : \sigma(o_{kj}^L) < \tau_{splat}\}$
- 3: $\mathcal{A}_k \leftarrow \mathcal{S}_k \setminus \mathcal{D}_k$
- 4: **if** $\mathcal{D}_k \neq \emptyset$ and $\mathcal{A}_k \neq \emptyset$ **then**
- 5: sample a donor in $\mathcal{A}_k$ for each row in $\mathcal{D}_k$ with weight proportional to $\sigma(o_{kj}^L)$
- 6: **for** each donor j' sampled m times **do**
- 7: compute redistributed opacity and scale for $m + 1$ copies using Eq. S5
- 8: write the adjusted donor back to j' and copy it into its m dead rows
- 9: **end for**
- 10: **end if**
- 11: **end for**
- 12: anneal coarse/fine proposal scales, batch sizes, and voxel size
- 13: **for** each primitive $c_k \in \mathcal{C}$ with $|\mathcal{S}_k| < S_{\max}$ **do**
- 14: compute primitive-local scores q_{kj} with Eq. S3
- 15: **for** each proposal batch $b \in \{\text{coarse}, \text{fine}\}$ **do**
- 16: sample unique parents j from $\mathcal{S}_k$ with probability proportional to q_{kj}
- 17: **for** each parent j **do**
- 18: propose $x^+ \leftarrow \mu_{kj}^L + \epsilon e^{s_{kj}^L} / (1 + \log(1 + d_{kj}))$ using the batch scale
- 19: accept x^+ with probability in Eq. S4
- 20: **end for**
- 21: **for** each accepted proposal with available free row r **do**
- 22: copy the parent parameters into row r , set $\mu_r^L \leftarrow x^+$, and set the row's primitive id to k
- 23: apply Eq. S5 to the parent and accepted child rows; update the same-primitive voxel counts
- 24: **end for**
- 25: **end for**
- 26: **end for**
- 27: reset $\bar{g}, \bar{\ell}$, and visit counts d for the next window

S1.6 Editing and animation details

The interactive editor operates on the handles exposed by the vocabulary representation. A user selects a region of the rendered scene with a brush stroke, which is projected back to the instances whose rendered splats fall inside the stroke. The selected instances are reassigned from their original primitives to one or more target primitives retrieved by an open-vocabulary text query. We preserve the selected instances' world-space positions, update their primitive indices, and resample rotation, scale, and opacity from the empirical distribution of instances already using the target primitive.

Both editing and animation use the same instance-level semantic features from the initial vocabulary assignment. We match to DINOtxt text-query embeddings with cosine similarity.

For animation, SCION keyframes the instance transform ϕ_i , so all splats inside the assigned primitive move together. We model wind as a traveling sinusoidal field that applies a coherent time-dependent transform in each instance's local frame. The rotation axis and amplitude are selected from the instance's semantic label and spatial location; instances outside $\{\text{flower}, \text{grass}, \text{leaves}\}$ remain static. Repeated structures therefore move as coherent units while still varying in phase and amplitude across instances.

The wind animation uses a bend axis perpendicular to the horizontal wind direction and world- z . The pivot of each instance is given by its lowest world- z splat with a global world- z offset of -0.1 . We use the same motion parameters for 3DGS and SCION: wind speed 1.0, wind strength 16.0° , sway frequency 0.5 Hz, turbulence scale 2.0, and fractal Brownian motion scale 50.

Algorithm S2 Default instance densification (one firing).

Require: instances $\{\phi_i^W\}_{i=1}^N$, accumulated gradients $\{\bar{h}_i\}$, thresholds $\tau_{instance}, \tau_g, \tau_s$, subdivide multiplicity M , caps $N_{max}, P_{max}, C_{max}$

- 1: $\bar{h}_i \leftarrow 0$ for non-finite values
- 2: $\mathcal{C} \leftarrow \{i : \sigma(\phi_i^W) < \tau_{instance} \vee \neg \text{finite}(\phi_i^W)\}$
- 3: remove all instances in $\mathcal{C}$ and compact the tensors
- 4: $\mathcal{G} \leftarrow \{i : \bar{h}_i > \tau_g \wedge \text{finite}(s_i^W)\}$
- 5: $\mathcal{P} \leftarrow \{i \in \mathcal{G} : e^{s_i^W} \geq \tau_s\}$
- 6: keep the top $\min(P_{max}, \lfloor (N_{max} - N)/(M - 1) \rfloor)$ parents in $\mathcal{P}$ by $\bar{h}_i$
- 7: **for** $i \in \mathcal{P}$ **do** $\triangleright$ subdivide large instances
- 8: **for** $m = 1, \dots, M$ **do**
- 9: sample $\epsilon_m \sim \mathcal{N}(0, I)$ and form child m with Eq. 9
- 10: set $\mathbf{q}_{i,m}^{W'} \leftarrow \mathbf{q}_i^W$ and $a'_{i,m} \leftarrow a_i$
- 11: **end for**
- 12: overwrite the parent row with child 1 and append children $2, \dots, M$
- 13: **end for**
- 14: $\mathcal{L} \leftarrow \{i \in \mathcal{G} : e^{s_i^W} < \tau_s\}$
- 15: keep the top $\min(C_{max}, N_{max} - N)$ instances in $\mathcal{L}$ by $\bar{h}_i$
- 16: **for** $i \in \mathcal{L}$ **do** $\triangleright$ clone small instances
- 17: sample $\xi \sim \mathcal{N}(0, I)$ and append a clone with Eq. 8
- 18: **end for**
- 19: reset instance gradient and loss accumulators to the new instance count

S1.7 Hyperparameters and training schedule

All reported SCION runs use the active vocabulary configuration in `config/`. We list only the parameters used by that stack; legacy strategy options retained for configurability are omitted.

Training schedule.

30k optimizer iterations; evaluation on one image every 1000 steps; full-image evaluation every 1000 steps; checkpoint save every 2000 steps; mixed precision disabled. Synthetic-NeRF scenes without a sparse point cloud are automatically seeded by a 3000-step splatfacto pretrain [Kerbl et al., 2023, Tancik et al., 2023] capped at 50k Gaussians, random scale 2.5, and opacity threshold 0.1.

Capacity.

Ours-Small uses $K = 128$, $S_0 = 4$, and $S_{max} = 128$, with the instance cap set per scene to 0.4 times the sparse point count. Ours-Large uses $K = 512$, $S_0 = 4$, $S_{max} = 128$, and all sparse points as initial instances. The global maximum instance count is 10^6 , the spherical-harmonic degree is 0, per-instance color offsets are disabled, and extra-compact checkpoint saving is enabled.

Loss and regularization.

The image loss uses the 3DGS photometric objective [Kerbl et al., 2023] plus VGG LPIPS [Zhang et al., 2018, Simonyan and Zisserman, 2014] with weight 0.05 evaluated every 4 steps. Splat and instance scale regularization weights are both 0.05. Splat and instance opacity regularization are disabled; the minimum instance opacity threshold is 5×10^{-3} .

DINO initialization.

DINO initialization is enabled and DINO training loss is disabled. We use DINOv3-ViT-L/16 at resolution 832, PCA dimension 64, feature dimension 64, voxel-hash lookup, $k = 2$ nearest neighbors, uniform per-view weighting, and no position-weighted DINO term.

Active density control.

The splat level uses the MH splat strategy with refinement from step 500 to 25k every 100 steps, opacity relocation threshold 0.01, birth acceptance threshold 0.5, and default gradient fallback threshold 3×10^{-3} . Primitive-level density control is disabled. The instance level uses the default strategy from step 1060 to 25k every 100 steps, with prune opacity 10^{-3} , grow gradient 10^{-4} , and grow scale 1.0.

Optimizer.

Learning rates are 5×10^{-5} for splat means, 2.5×10^{-3} for DC color, 2×10^{-2} for splat opacity, 2×10^{-3} for splat scale, 10^{-3} for splat rotation, 7×10^{-5} for instance position, 7×10^{-4} for instance rotation, 5×10^{-4} for instance scale, 2×10^{-2} for instance opacity, 10^{-4} for camera optimization, and 2×10^{-3} for the bilateral grid. The scene learning-rate scale is automatic.

Discriminator.

The projected discriminator is enabled after a 10k-step warmup with loss weight 8.6×10^{-3} , learning rate 1.06×10^{-5} , $\beta_1 = 0$, update frequency 8, one discriminator step per update, patch size 64, 32 patches, instance noise 0.05, no spectral norm, no instance norm, and no R1 penalty. ADA uses target 0.8, speed 0.01, interval 4, and maximum probability 0.9. The GAN loss is smoothed BCE with label smoothing 0.1, using the two finest VGG feature scales.

Dataset settings.

MipNeRF360 and Tanks & Temples use the default 30k-step schedule. Synthetic-NeRF uses a white background.

S1.8 Compute resources

Most reported SCION runs and the rerun matched-storage baselines were trained on NVIDIA V100 GPUs; selected runs used a single workstation with one NVIDIA RTX 5090 GPU (32 GB VRAM) and an Intel Core Ultra 9 285K CPU (24 cores, up to 5.5 GHz). Peak VRAM stayed within 15.4 GB across all runs, so a single GPU with 16 GB of memory is sufficient to reproduce the reported results and multi-GPU training is not required. Per-scene wall-clock time at the standard 30k-iteration schedule was between 1.47 and 2.18 hours on a V100 and at most 1 hour on the RTX 5090.

S2 Additional experimental results

S2.1 Full reconstruction results

Tab. S1 reports the complete benchmark table at each method’s native storage cost. These numbers are included for traceability, but the main text emphasizes the matched-storage setting because SCION is designed to reuse primitives rather than maximize per-pixel memorization.

Table S1: **Full reconstruction results.** PSNR, SSIM, LPIPS, and on-disk model size are reported at each method’s native operating point. Baseline results are re-reported from the corresponding papers unless otherwise noted; we selectively reproduced and cross-checked reported methods.

Method	MipNeRF360				Tanks & Temples				Deep Blending				Synthetic-NeRF			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	MB $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	MB $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	MB $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	MB $\downarrow$
3DGS	27.46	0.812	0.222	750.9	23.69	0.844	0.178	431.0	29.42	0.899	0.247	663.9	33.80	0.970	0.031	68.46
MCMC-3DGS	29.89	0.900	0.190	—	24.29	0.860	0.190	—	29.67	0.890	0.320	—	—	—	—	—
Scaffold-GS	27.50	0.806	0.252	253.9	23.96	0.853	0.177	86.50	30.21	0.906	0.254	66.00	33.41	0.966	0.035	19.36
Compact3DGS	27.08	0.798	0.247	48.80	23.32	0.831	0.201	39.43	29.79	0.901	0.258	43.21	33.33	0.968	0.034	5.54
Compressed3DGS	26.98	0.801	0.238	28.80	23.32	0.832	0.194	17.28	29.38	0.898	0.253	25.30	32.94	0.967	0.033	3.68
ContextGS (low-rate)	27.62	0.808	0.237	12.68	24.20	0.852	0.184	7.05	30.11	0.907	0.265	3.45	32.79	0.965	0.040	1.01
LightGaussian	27.00	0.799	0.249	44.54	22.83	0.822	0.242	22.43	27.01	0.872	0.308	33.94	32.73	0.965	0.037	7.84
CompGS (low-rate)	26.37	0.778	0.276	8.83	23.11	0.815	0.236	5.89	29.30	0.895	0.293	6.03	—	—	—	—
SOGS	26.56	0.791	0.241	16.70	23.15	0.828	0.198	9.30	29.12	0.892	0.270	5.70	31.37	0.959	0.043	2.00
HAC++ (low-rate)	27.60	0.803	0.253	8.34	24.22	0.849	0.190	5.18	30.16	0.907	0.266	2.91	33.03	0.966	0.039	0.88
Nexels	27.35	0.802	0.201	160.0	23.55	0.841	0.155	160.0	—	—	—	—	—	—	—	—
BBSplat	26.98	0.783	0.231	260.0	23.83	0.854	0.147	260.0	—	—	—	—	—	—	—	—
Ours-Small	22.00	0.599	0.432	1.20	18.61	0.628	0.416	1.55	24.02	0.807	0.442	0.73	24.39	0.896	0.105	0.16
Ours-Large	23.05	0.646	0.348	3.17	19.51	0.674	0.330	4.04	24.55	0.812	0.410	2.01	26.31	0.917	0.072	0.47

S2.2 Capacity ablation

Tab. S2 sweeps each of the four capacity knobs while the others stay fixed at the Ours-Large default.

References

Ian J Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative Adversarial Nets. *Advances in neural information processing systems*, 27, 2014.

Table S2: **Capacity sweep.** Each group sweeps one knob; the others are held at Ours-Large defaults.

Knob	Value	PSNR $\uparrow$	SSIM $\uparrow$	MB $\downarrow$	bits/G $\downarrow$	# Total Splats
K	64	21.44	0.532	1.36	6.6	1.67M
	128	21.65	0.545	1.45	6.7	1.74M
	256	22.01	0.564	1.60	7.6	1.70M
	512	22.40	0.584	1.92	9.1	1.69M
	1024	22.61	0.603	2.52	12.2	1.65M
S_0	2	21.91	0.549	1.55	8.5	1.47M
	4	22.01	0.564	1.60	7.6	1.70M
	8	22.23	0.580	1.66	6.8	1.96M

Tero Karras, Miika Aittala, Janne Hellsten, Samuli Laine, Jaakko Lehtinen, and Timo Aila. Training Generative Adversarial Networks with Limited Data. *Advances in neural information processing systems*, 33:12104–12114, 2020.

Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM Transactions on Graphics*, 42(4), July 2023.

Hyunjin Kim, Haebeom Jung, and Jaesik Park. Metropolis-Hastings Sampling for 3D Gaussian Reconstruction, June 2025.

Axel Sauer, Kashyap Chitta, Jens Müller, and Andreas Geiger. Projected GANs Converge Faster. *Advances in Neural Information Processing Systems*, 34:17480–17492, 2021.

Johannes Lutz Schönberger and Jan-Michael Frahm. Structure-from-motion revisited. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, Francisco Massa, Daniel Haziza, Luca Wehrstedt, Jianyuan Wang, Timothée Darcet, Théo Moutakanni, Leonel Sentana, Claire Roberts, Andrea Vedaldi, Jamie Tolan, John Brandt, Camille Couprie, Julien Mairal, Hervé Jégou, Patrick Labatut, and Piotr Bojanowski. DINOv3, 2025.

Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.

Matthew Tancik, Ethan Weber, Evonne Ng, Ruilong Li, Brent Yi, Terrance Wang, Alexander Kristoffersen, Jake Austin, Kamyar Salahi, Abhik Ahuja, et al. Nerfstudio: A Modular Framework for Neural Radiance Field Development. In *ACM SIGGRAPH 2023 Conference Proceedings*, pages 1–12, 2023.

Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The Unreasonable Effectiveness of Deep Features as a Perceptual Metric. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 586–595, 2018.